3 Numerical methods and comparison protocol

Section 2 gave a sparse, symmetric positive-definite system $Ax = b$ for the free displacements. We now compare ways to solve it. Direct methods factor the matrix; iterative methods improve a displacement estimate. Each iteration approaches the same static equilibrium, not a later time.

3.1 Solution methods

For an iterate x_k, the residual $r_k = b - Ax_k$ measures the remaining force imbalance. We write $P = \mathrm{diag}(A)$ for the diagonal stiffness entries. The following methods offer different ways to reduce that imbalance [1].

Cholesky

Cholesky uses positive definiteness to write $A = LL^T$, with L lower triangular. Solving $Ly = b$ and then $L^T x = y$ gives the displacement. We use it in the small example because it directly exploits our matrix's structure; y is only an intermediate vector.

LU and the direct reference

Pivoted LU factors a row-permuted matrix into lower and upper triangular factors, then solves the resulting triangular systems. Sparse LU provides x_{ref}, a comparison solution on the same mesh. It is *untimed*, not a competitor in the iterative timing table. The small example also forms $A^{-1}b$ for illustration, not as the recommended method.

Jacobi

Jacobi corrects every component using only the previous iterate: $x_{k+1} = x_k + P^{-1}r_k$. We try it as a simple baseline with inexpensive updates. Positive definiteness alone does not guarantee its convergence.

Damped Jacobi

Damped Jacobi takes a smaller correction, $x_{k+1} = x_k + \omega P^{-1}r_k$. We use $\omega = 0.5$ to test whether reducing the step improves convergence. This weight is not guaranteed to work for every positive-definite matrix.

Gauss–Seidel

Gauss–Seidel updates components sequentially, immediately reusing the newest values. Comparing it with Jacobi tests the benefit of using fresh information. It converges for our positive-definite system in exact arithmetic, though it may still require many sweeps.

Steepest descent

Steepest descent moves along the residual, choosing the step that minimizes $\frac{1}{2}x^T Ax - b^T x$ in that direction: $x_{k+1} = x_k + \alpha_k r_k$, where $\alpha_k = (r_k^T r_k)/(r_k^T A r_k)$. We include it to test whether choosing the step from the matrix improves on fixed-step corrections.

Conjugate gradients (CG)

CG combines the residual with previous search information rather than starting afresh at every step. It minimizes the same quadratic energy over an expanding search space. We expect it to suit this problem because it exploits both symmetry and positive definiteness [1].

Diagonal preconditioned CG (PCG)

PCG scales the residual by P^{-1} before constructing its search direction. We test whether accounting for different diagonal stiffnesses reduces the work needed by CG. The scaling preserves the solution, but its benefit must outweigh its cost. SciPy receives the inverse action P^{-1} as its `M` argument [2].

GMRES

GMRES chooses the smallest Euclidean residual in a space built by repeated multiplication by A. We include this more general method as a comparison with CG. Our unpreconditioned run uses a restart length equal to the number of free unknowns: full GMRES, whose stored basis and orthogonalization can become expensive on larger meshes [3].

MINRES

MINRES also minimizes the residual, but uses symmetry to update the solution through a short recurrence. It provides a symmetry-aware comparison with GMRES and CG. We do not need it because of indefiniteness: our supported matrix is positive definite [1].

3.2 Common stopping and timing rules

All eight iterative methods use identical A, b, start from $x_0 = 0$ and have a 20 000-iteration budget. For nonzero b, acceptance requires the independently recomputed relative residual

$$\rho(x) = \frac{\|b - Ax\|_2}{\|b\|_2} < 10^{-8}, \tag{1}$$

where $\|\cdot\|_2$ is the Euclidean norm. One iteration is a full stationary sweep or a Krylov step; GMRES counts inner Arnoldi steps, not restart cycles. Different work per step prevents ranking speed by counts alone. The solver handles $b = 0$ by returning $x = 0$ immediately; the current benchmark's outer relative-error calculations require nonzero denominators.

CG and PCG use relative tolerance 10^{-8} and absolute tolerance zero. MINRES uses native tolerance zero and an explicit true-residual stopping callback. GMRES uses the legacy callback convention for inner-step counting; its history contains Arnoldi residual estimates, unlike the explicitly computed histories of the other methods. Final acceptance always uses Eq. (1). We retain native status and acceptance separately, including breakdowns and budget exhaustion. Stationary and steepest-descent runs stop if ρ is nonfinite or exceeds 10^{10}. Failed-run times are not times to the requested accuracy.

We retain three timed runs and report their minimum, collecting histories separately. Timings include iterative setup and required stopping checks, including MINRES's per-step true residual. They exclude assembly, conditioning diagnostics, the LU reference and final independent checks. Thus they are *setup-inclusive iterative times*, not separate setup and solve measurements. Machine and software details accompany the data; small timing differences do not establish general rankings.

3.3 Accuracy and validation

We report time, iterations, residual and the relative reference difference

$$e_{\mathrm{ref}}(x) = \frac{\|x - x_{\mathrm{ref}}\|_2}{\|x_{\mathrm{ref}}\|_2}. \tag{2}$$

We check the reference's residual too. Poor conditioning can leave appreciable displacement error despite a small residual. These are solver-reliability checks; load perturbations at fixed A instead test input sensitivity, with magnitude and direction recorded.

Solver agreement does not establish continuum accuracy. Refinement must preserve the physical case and compare matching locations or observables. Section 4 also checks prescribed displacements and force/moment balance using Section 2's support reactions, not free-row residuals. Its separate exact affine test uses roller supports, not the clamped benchmark.

References

[1] Yousef Saad, *Iterative methods for sparse linear systems*, second ed., SIAM, 2003.

[2] SciPy developers, *scipy.sparse.linalg.cg*, `https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.cg.html`, API documentation.

[3] ———, *scipy.sparse.linalg.gmres*, `https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.gmres.html`, API documentation.